

Predicting the Predictor: When Does AI Bond Pricing Miss? Regime-Based Error Analysis and Residual Correction Using BondDroid and TRACE

SIADS 696 (Milestone II)

Transaint Gau (tgau@umich.edu) , Christian Goelz (chgoelz@umich.edu), Carter Pasternak (pasterna@umich.edu)

Introduction and Overview

Artificial Intelligence (AI) is a field that progresses day-by-day, with seemingly near infinite uses being developed every moment for better or for worse. This ranges from the most statistical fields, such as mathematics and sciences, to the least statistical, such as art, and within this range lies the field of finance. Within the field of finance, an enterprise tool called BondDroid, an AI corporate bond pricing system, has been developed and operational since around the year 2018. BondDroid is a tool that generates bid and ask estimates for bonds based upon liquidity signals and other related pricing information, intended to assist institutional participants in fixed-income markets, from people like investors, analysts, to portfolio managers. Despite this reliance on BondDroid, limited research has been done on the reliability, accuracy, and conditions in which it fails.

Our purpose is to address this lack of knowledge in BondDroid's pricing accuracy in differing bond and market conditions. In particular, our project revolves around BondDroid's midpoint estimates as true, the realized transactions that occur, and when systematic errors occur in its predictions. This problem may have major implications for the major market participants mentioned previously, who may rely on pricing estimates for valuation and transactions. Identifying when BondDroid performs poorly, or when it performs well, aids in building trust in the system and confidence in AI-assisted pricing systems as a whole and may further highlight where further human oversight or model adjustments are needed to ensure errors are not reproduced. This question creates broader discussions of machine learning (ML) and explainability and reliability, not only in financial market modeling, but also in other systems that rely on ML modeling systems.

This line of questioning is inspired by the use of ML and AI in business spaces, especially the use in the financial sector, alongside the lack of sufficient evaluations or research into errors in enterprise-level bond-pricing systems such as BondDroid. For example, past research has gone into applied regression, neural networks, or gradient boosting methods as main approaches to forecast bond prices, returns, or even measure illiquidity. However, unlike these papers, our project does not intend to solve a new prediction method, but instead model BondDroid's own errors. With this, the main question does not become whether ML methods can predict bond prices, as this has been discussed countless times before, but it is instead what conditions may cause a fully realized operational model to fail.

To address our question, we combine both unsupervised and supervised learning methods, using BondDroid pricing signals matched with actualized TRACE transaction data. Unsupervised learning methods are used here to predict pricing and liquidity regimes within BondDroid data, and to analyze whether or not pricing differs systematically among environments, further using supervised learning. We approach unsupervised learning using K-means to develop clusters, with

Gaussian Mixture Methods (GMMs) to compare overlapped clusters, and HDBSCAN to understand density-based clustering. Cluster validities are measured on K-means using silhouette scores, Interia (Elbow), and Davies-Bouldin to verify validity. Dimensionality reduction is applied, using UMAP for visualization of clusters, retaining the original features.

Supervised learning methods are used to model BondDroid pricing errors relative to the realized TRACE trades. The primary target is the absolute pricing error between BondDroid midpoint estimates and the TRACE transaction price. Our models for this include regularized linear regression (Ridge and Lasso), Random Forest, and gradient boosting methods (XGBoost, LightGBM) for a wide range of methods to test what is best. These are used to predict the absolute pricing error, which is a benchmark number assumed to be the baseline, where BondDroid pricing is correct. These methods are then applied directly to regimes established within unsupervised learning to determine whether the reliability of BondDroid varies within different market environments. The feature strength of the best models are examined with respect to the MAE, and this value is used as predictive weight for BondDroid.

Through our analyses, we found several important factors going into BondDroid and its predictions. Using clustering methods, we have found four pricing regimes that seem to be related to how BondDroid predicts prices and drives error. These regimes each have a different level of error in pricing, and thus have different weights on features going into modeling. Alongside our discoveries through clustering, we discovered that features related to each cluster vary by group, including staleness, liquidity, and other bond descriptors like coupon. Outside of clusters, we found that error is seemingly driven in other ways, instead of being related solely to one main group of features; it is driven by a mix of factors like term length, liquidity, and other bond descriptors, however, at a lower rate than inside of clusters. These findings show that while BondDroid may work on an enterprise level, there are factors within bonds that may alter or impact predictions unknowingly at varying rates.

Related Works

While researching, we found several similar papers to our own. However, no project that discusses enterprise bond systems like BondDroid currently exists. The closest project to ours was a working paper by Daly et al. (2024) analyzing TRACE and CDS data from 2018-2023, for 750 liquid investment-grade bonds from the LQD ETF. This paper tests several methods for bond pricing and trading strategies, including OLS, penalized regression, Random Forest, LSTM models, and gradient boosting. While our project uses similar models such as Random Forest, gradient boosting, and penalized regression (Lasso and Ridge), our project instead evaluates commercial pricing systems available for use, while Daly et al. are researching new methods to predict bond pricing and trading strategies.

Another paper developed by Feng et al. (2025) applies ML methods to forecast individual corporate bond returns utilizing historical bond predictors, this includes factors such as spread and macroeconomic variables like S&P 500 index returns and GDP. The study evaluates whether methods such as nonlinear machine learning models may predict and outperform standard methods in predicting bond returns, and how performance changes based on market conditions. Although this paper does discuss predictions based upon market conditions, our project instead focuses on BondDroid's performance based on traditional bond features and regimes present within bonds. Alongside this, we intend on modeling the performance of BondDroid itself, and

not a novel model, unlike Feng et al.

A third related study by Cabrol et al. (2024) applies ML techniques to predict corporate bond illiquidity using features such as bond-level characteristics, trading variables, and market information. The study aims to discover whether ML improves the prediction accuracy of liquidity conditions and whether ML is able to identify factors associated with liquidity. This differs from our own research in several ways, while both may look at features and how they influence others, Cabrol et al. is looking at how features may identify liquidity. For us, we are aiming at how features impact accuracy in pricing, which may also include liquidity in pricing accuracy. Alongside this, we intend to look at clusters, in the hopes of identifying how pricing regimes alter or affect pricing accuracy in BondDroid.

Although the three models analyze bonds using machine learning methods, including modeling techniques similar to what we intend on using, our project varies in several fundamental ways. To begin, our analysis is based on the accuracy of pricing in BondDroid, an already developed ML model aimed at predicting bid and asking prices in bonds. On the other hand, these three research projects develop their own models, either looking at novel pricing models themselves or new methods to predict factors such as liquidity. While these papers each have their own ways to investigate bonds, our research is aimed directly at an enterprise system. Thus, while we hold several similarities in models or methods used, our research does not directly build upon their discoveries, but rather aids BondDroid and those using it or planning to.

Data Sources

Our project uses two data sources for our analysis. First, we utilize *Top Liquid Bond Prices & Liquidity Indicators*, a dataset generated with BondDroid and obtained from the Dewey Data website (7 Chord, 2025). This data is supplied in the form of a CSV spanning from October 5th, 2023, to March 18th, 2025. This dataset is large, spanning nearly 3 GiB of data totaling 36,852,929 records without engineering. While this dataset is normally paid, it is provided free for University of Michigan students and professors. From this dataset, we maintained all columns and rows upon downloading the data. All variables here contain important information; however, most importantly, we are looking at identifiers for trades, bid-ask prices, and liquidity identifiers.

The other source of data we utilized was *FINRA TRACE Enhanced* transaction-level data, provided by WRDS (FINRA, 2026), which is free for University of Michigan students and professors, much like our other source. This source is much larger with the enhanced dataset totalling 95.03GiB with over 450 million records in total; from this dataset, we focused on transaction-level data, such as execution time, price, yield, and volume for corporate bonds. Along with this, we intend to look at identifiers to match with BondDroid data. As per WRDS, this data spans from January 1st, 1900, and was last updated May 14th, 2026.

Feature Engineering

Our datasets required substantial preprocessing due to their aforementioned size and complexity. The primary objective was to align TRACE transactions alongside the most recent BondDroid observations available to the trade. Because TRACE identifies bonds using CUSIP identifiers rather than Financial Instrument Global Identifiers (FIGI), which BondDroid uses, a custom

FIGI-to-CUSIP reconciliation process was developed using FISD data. BondDroid and FISD records were standardized using issuer names and maturity dates. Issuer names were first normalized by removing corporate suffixes and formatting errors or inconsistencies. Bonds were then initially matched using normalized issuer names and maturity dates. The remaining unmatched observations were then matched using FISD CUSIP issuer names alone. When matching was complete, duplicate FISD records were removed, and match quality was evaluated, a master lookup table was created linking FIGI and CUSIP identifiers.

Following reconciliation, BondDroid and TRACE data were merged into one unified panel dataset, using the now-established FIGI-to-CUSIP mappings. TRACE transactions were aligned to BondDroid observations using intraday timestamps, based on when the most recent BondDroid quote was available before execution of the trade, using an ASOF join.

After the aligned panel was constructed, additional processing was performed to prepare data for machine learning. The main target for supervised learning (`target_abs_error`) was defined as the absolute difference between the BondDroid midpoint estimate and the corresponding TRACE transaction price. This allows us to measure the magnitude of the pricing error, regardless of direction, higher or lower than the trade price, and serves as an indicator of BondDroid's pricing performance.

Missing values in the dataset were addressed using imputation and feature-specific transformations. Missing values in trading activity, such as the average daily traded counts, were replaced with zeros. Missing pricing variables, including BondDroid midpoint yield, BondDroid midpoint price, duration, and convexity, were imputed with backward-fill and forward-fill within each bond's unique FIGI history to reduce the impact of missing values on ML outcomes. This ensures that quotes are informative, even if values are unavailable. Following imputation, features were engineered to capture market conditions and trading behavior; options-adjusted spread (OAS) and spread-over-treasury (SOT) midpoint measures were calculated by averaging the bid and ask observations for each variable. Because institutional transactions can vary in size, especially if companies are larger or smaller, trade volume was then transformed using a log-scale, reducing skewness in data and limiting the influence of extreme observations. Categorical variables, including market segment, geographic region, BondDroid sector, and trigger type, were then converted into numerical values for use in ML systems.

Because the final aligned dataset contains more than 13 million observations, we used Polars to perform preprocessing and save time. Polars allowed for more efficient feature engineering and transformation of our datasets, alongside letting us overcome memory limitations present with Pandas. After the completion of preprocessing, the resulting ML-ready dataset was converted to a Pandas DataFrame for modeling, visualization, and evaluation, and spanned 8,509,326 records once clean.

The full list of features is included in the appendix.

Unsupervised Learning

Unsupervised learning began by preparing data and further cleaning of prior prepared data. To ensure our clusters did not pull any unnecessary data and weight, we determined redundancy within data using Spearman correlations on the candidate features and selected the best variables.

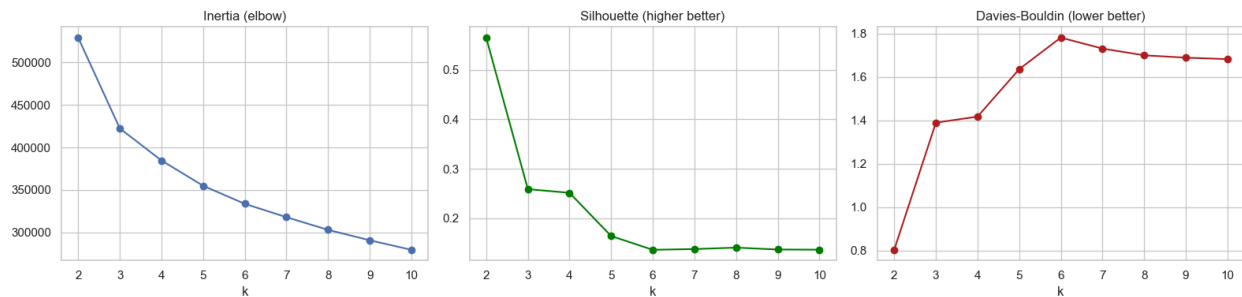
Doing this, we avoided double-weighting dimensions in distance metrics and thus allowed for better evaluation of clusters. Variables that were kept describe distinct dimensions such as liquidity, credit, and rate-level dimensions, and none of these were rank-identical, proving that our choices were made justly.

After determining the redundancy of variables, we scaled features using a robust scaler centered around the median value. This was needed as we used K-Means to determine clusters, alongside GMM and HDBSCAN to corroborate results, and a traditional standard scaler may keep outliers, which may cause high sensitivity in determining these clusters. As binary features may also interfere, we also removed our one-hot encoded features, such as sectors, industry, and region, so as not to dominate the Euclidean distance. These features were then examined to ensure that we account for variance in features and that no features include little or no variance, as this would limit cluster detection.

Post feature engineering, K-Means was used to determine clusters within the data, with three distinct models being assessed. For this, we tested inertia (elbow), Silhouette, and Davies-Bouldin methods to determine some clusters to analyze. While clusters were generated using the full 8.5M rows of data, models were assessed on a random selection of 100k rows to save computational power.

Figure 1.

Inertia (Elbow), Silhouette, and Davies-Bouldin K-Means Testing



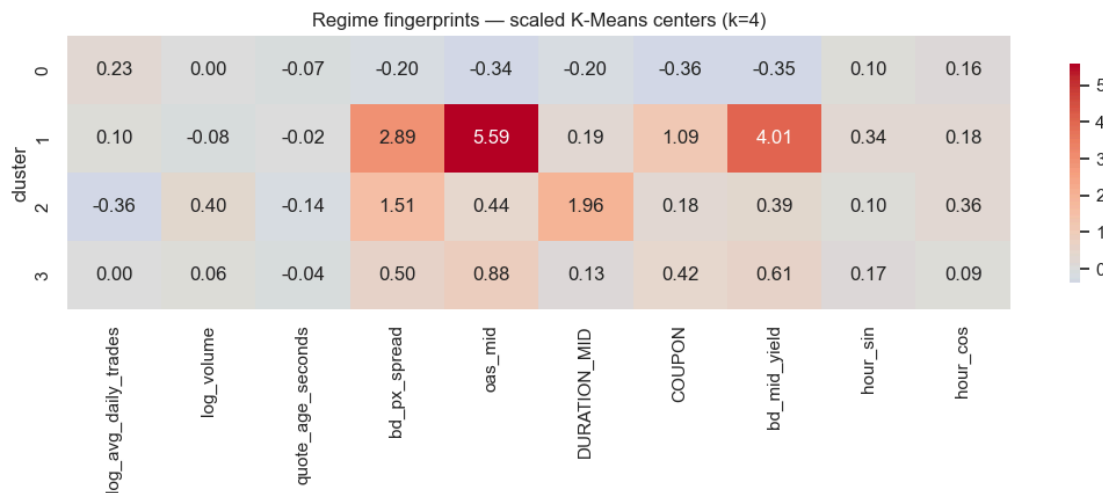
Note: K-Means values were assessed for each model using a random sample of 100k rows of data, sweeping over $K = 2$ -10. These three models were chosen as they are complementary signals to one another.

Inertia elbow modeling flattens around $K = 3$ -4, Silhouette maximizes around $K = 2$, and Davies-Bouldin is lowest around $K = 2$ (Fig. 1). While $K = 2$ seemed to indicate the most dominant K-Means, $K = 4$ yielded four economically distinct regimes, with no micro clusters, thus we used $K = 4$ as the primary segmentation. GMM was then used as a metric to evaluate elliptical clusters, alongside HDBSCAN as a method to measure clusters based on density. Using the same random sample of 100k records as in model assessment, we analyzed the Adjusted Rand Index (ARI) of GMM, resulting in a value of 0.39, indicating the clusters are not randomly generated, although they may miss some boundaries, corroborating our selection of 4 clusters found using K-Means. HDBSCAN was then used to examine cluster density, through this it was indicated that there are two dense clusters, with around 11% of the data representing noise. Analyzing the non-noise datapoints, we found an ARI of 0.039, indicating these clusters are

closer to random than they are true clusters, thus pushing us to use four clusters as indicated by K-Means and corroborated by GMM.

Figure 2.

Regime Fingerprints: Clusters by K-Means Center

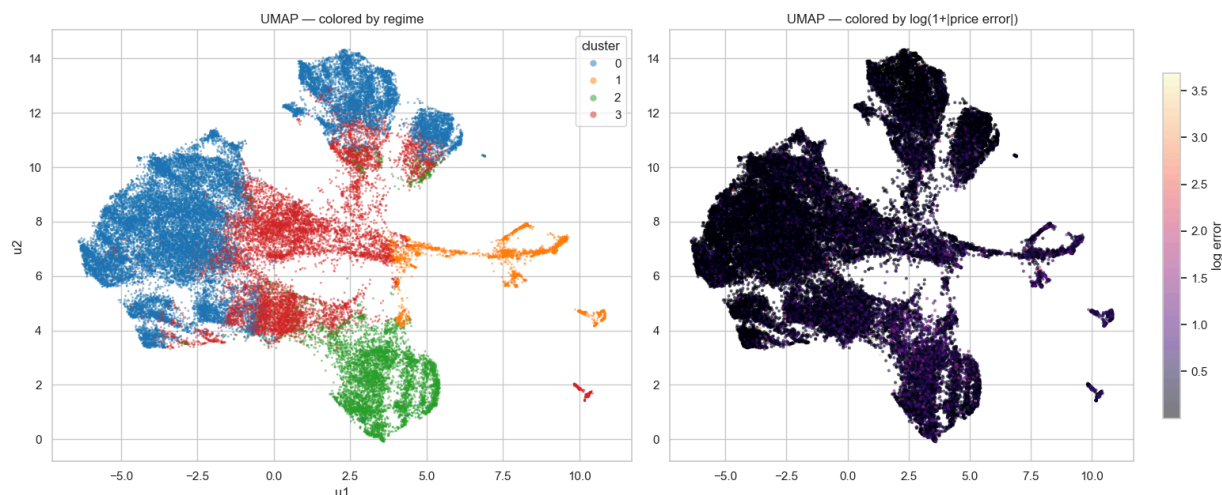


Note: Clusters are evaluated using K-Means centers of each feature. These features only include values that are numerical in nature, such as spread, yield, hour, trades, or duration.

Following the analysis of cluster noise and assignments, the clusters themselves were analyzed. Regime fingerprinting was analyzed to determine the pieces that determine what features dominate and drive shaping in a cluster. These features analyzed only included numerical variables in clusters, yet it shows that variables may still highly correlate with a cluster itself, such as `oas_mid` in cluster 1, or `log_volume` in cluster 2 (Fig. 2).

Figure 3.

UMAP Representation of Clustered Data



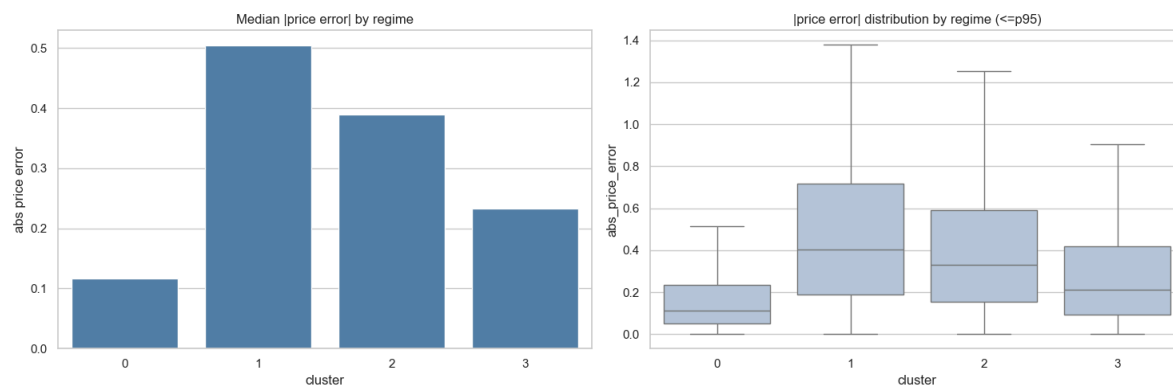
Note: Clusters were displayed using UMAP embedding on a 50k sample of data on a 2D plane. This differs from our cluster analysis, which is over 10D of data, and as such, only serves as a representation.

Another random sample of data was dimensionally reduced using UMAP embedding and displayed with both color by regime and log of price error (Fig. 3). This figure further demonstrates that while clusters were separated, there is a slight amount of noise, as seen in the cluster colored graph. And among the entire sample of data, price errors tend to gather within clusters, indicating a dispersion of error among clusters.

The results found from unsupervised learning methods were then used within supervised learning to examine pricing errors generated by BondDroid, within and outside of clusters.

Figure 4.

Absolute Pricing Error by Regime



Note: Pricing errors represent the entire sample of data (8~million rows), and use the engineered `abs_price_error` feature designed in our data processing steps for analysis.

Supervised Learning

Due to the findings of four usable clusters (Fig. 1) and pricing error being dependent on the cluster (Fig. 4), the goal going into supervised learning was to generate an understanding of how and why BondDroid gets these clusters wrong. To begin with this task, the completed ML-ready dataset was first merged with the clustering labels using an as-of join on trade time and BondDroid estimation time by the FIGI classifier. This merge was then checked with row counts and null counts to ensure no missing values or incorrect clusters were assigned, and then trades with no cluster were dropped to ensure we only measure trades with a cluster.

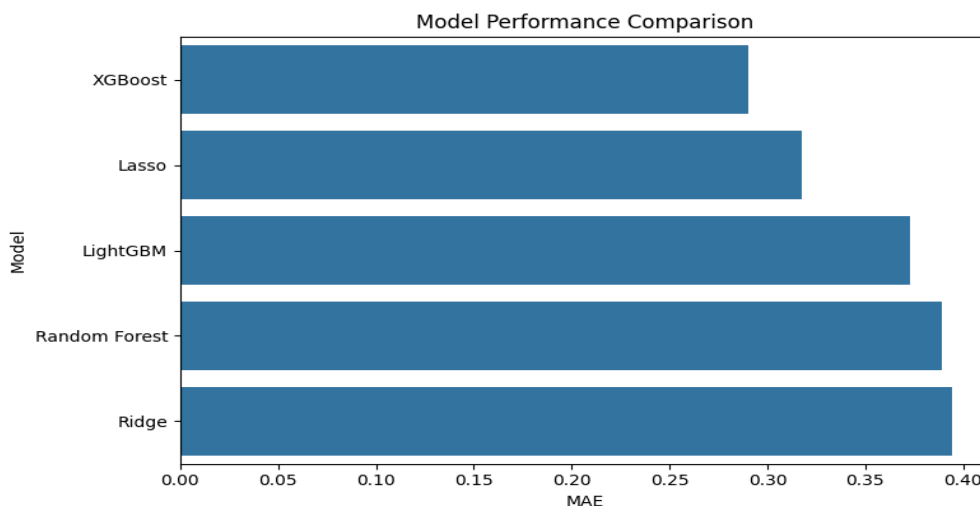
With the datasets confirmed to be merged, new features were engineered to ensure supervised learning would perform well. This included changing trade-time to hour of the day, and day of the week, the maturity of the bond was changed to years to maturity by subtracting the maturity date from trade time and dividing by 365.25 days to account for leap years, and dummy variables were applied to time since the last trade. The dataframe was then sorted in timestamp order to prepare for a train-validate-test (TVT) split using time to prevent leakage. Performing a TVT split was done using a quantile calculation on the timestamp, with 70% of the data allocated to

training, and 15% allocated to both validation and testing.

Following the creation of the TVT split, these subsets of data were used with several different models in order to determine the best approach to understanding failure in BondDroid pricing. For this, we included three families of modeling: traditional linear regression with Lasso and Ridge regression, ensemble methods through bagging with Random Forest, and boosting with XGBoost and LightGBM. Mean absolute error (MAE) was used as an evaluation metric for these models to ensure the predicted error was closest to the target, which was the absolute pricing error by BondDroid. Ensuring that MAE was low ensured that our model was close to BondDroid's predictions, and as such, our features and their importances are a good estimation of how BondDroid performs.

Figure 5.

Model Performance (MAE) by Model Used

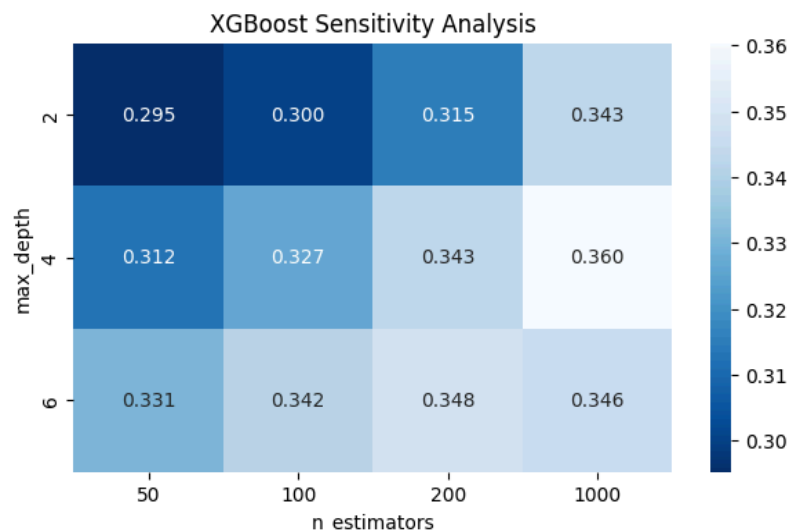


Note: Models were assessed on several different hyperparameters, and the MAE from the best-performing hyperparameter is displayed. Model performances ranked lowest to highest MAE: XGBoost 0.290, Lasso 0.317, LightGBM 0.331, Random Forest 0.373, Ridge 0.394

With the MAE ranked (Fig. 5), we were able to analyze the best-performing model. In this case, while all models performed relatively similarly using MAE as a metric, XGBoost stood out with the lowest average of absolute error at 0.290, making it our model of choice. Proceeding with XGBoost, we decided to perform sensitivity testing on the model, identifying MAE with regard to hyperparameters like the number of estimators and depth.

Figure 6.

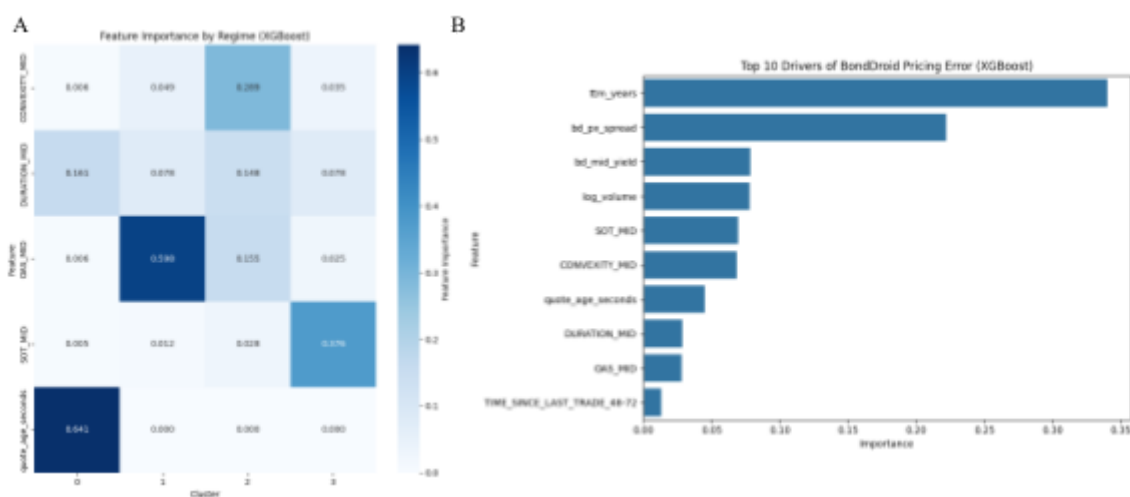
XGBoost Sensitivity Analysis by MAE on Hyper Parameters



Post determination of the best model, we decided to perform sensitivity analysis on XGBoost to determine the effectiveness of the model and the strength of the hyperparameters. By altering the max depth and number of estimators in the model, we found a varying degree of success determined by the MAE (Fig. 6). In our analysis, it appears that the best performance occurred with a depth of two, meaning there are only two splits per tree. While using a max depth of two, we saw the best performance from fifty estimators (decision trees), while one-hundred was close behind. While this was the best performance, we saw similar results from higher depths with lower decision tree counts, which is commonly seen in noisy data. The downside to this, however, is that training models with higher depth often takes much longer, even with a smaller number of decision trees in a model. Because of this, while there may have been a potential to see better performance from lower amounts of decision trees and a higher depth, the fact that this training would take longer with our amount of data was not worth the trade-off. Alongside this, due to the fact a low depth is often better for larger counts of variables to prevent bias and overfitting, this solution is better.

Figure 7.

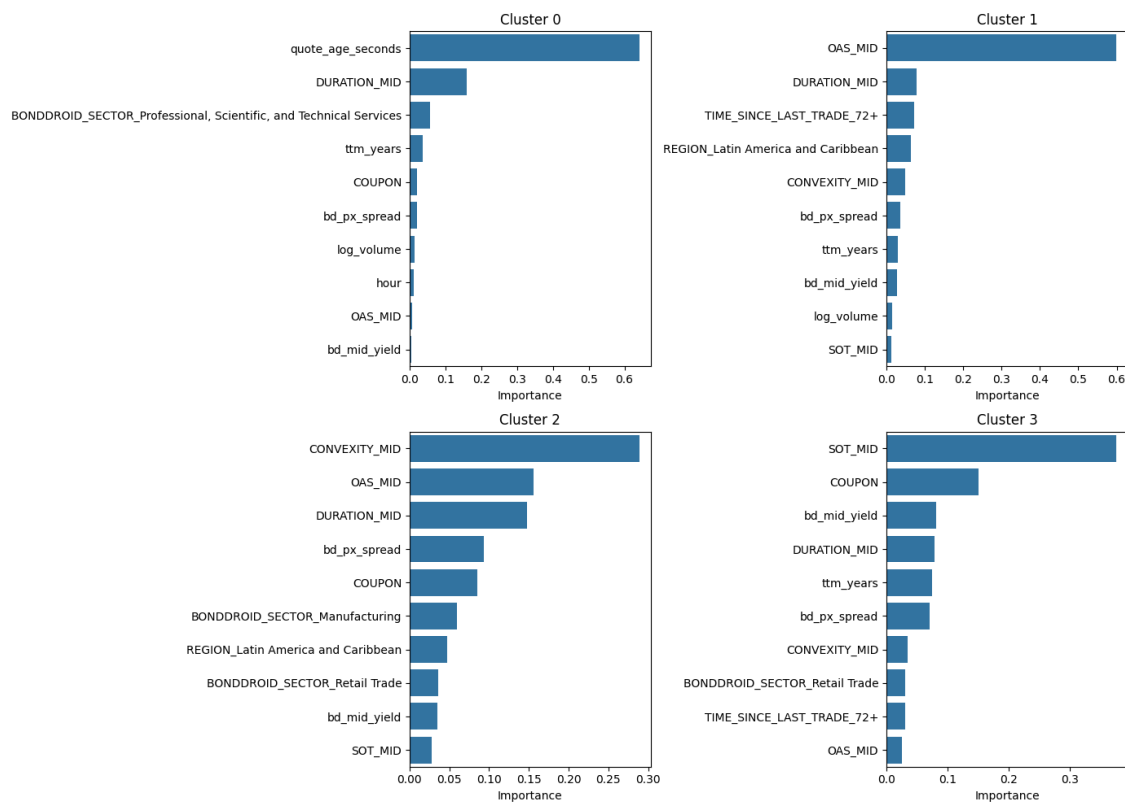
Feature Importance in XGBoost Modeling



Note: Feature importance is determined by gain, i.e. average improvement in modeling prediction. Subplot A represents the feature importances separated by the four pricing regimes shown as the top average features. Subplot B represents the top ten feature importances for the entire dataset, not split by regime.

Figure 8

Top 10 Feature Importances by Cluster



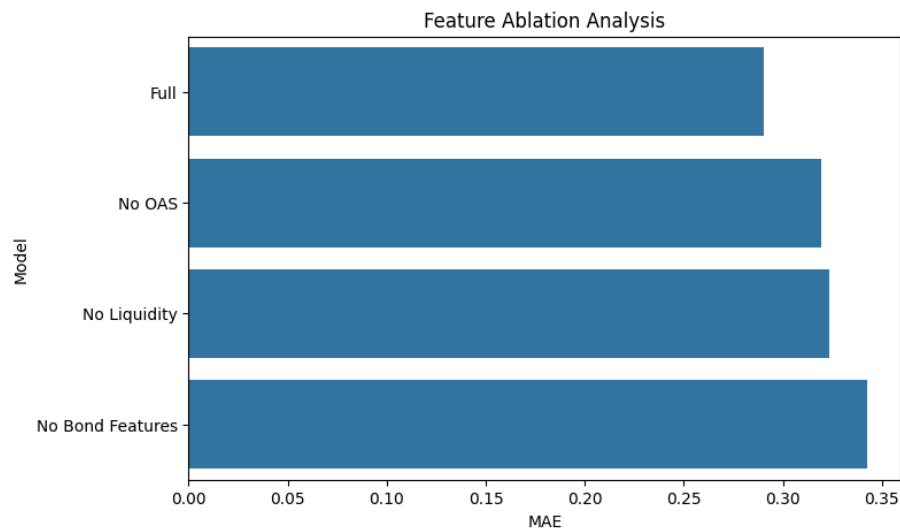
Note: Feature importance is determined by gain, i.e., average improvement in modeling prediction. The top ten features for each cluster are shown.

While we have previously seen errors appear to differ across clusters (Fig. 4), we also wanted to examine how features play different roles in modeling, both inside and outside clusters. To do so we examined the feature importance by gain using the best XGBoost model, both with and without clusters. Inside clusters, we saw that there are seemingly vastly different features that play important roles in modeling (Fig. 7A), with `quote_age_seconds` seemingly dominating cluster 0 (0.641), `OAS_MID` dominating cluster 1 (0.598), `CONVEXITY_MID` at the top of cluster 2 (0.289), and `SOT_MID` at the top of cluster 3 (0.376) (Fig 7A). While this is the case, some features appear to have more of an equal weight in multiple clusters, including

DURATION_MID with nearly identical feature importance in clusters 0 and 2, at 0.161 and 0.148, respectively. On the other hand, features appear to have different importance when examined without clusters (Fig. 7B). When clusters are absent from the data, feature importance has a much lower strength, with the highest gain being around 0.35. Alongside this, the features vary from what was seen in clusters, with the top feature being ttm_years, next to bd_px_spread as the second most important feature.

Figure 9

Feature Ablation Analysis in XGBoost



Note: Features were removed in groups based on their description and feature importance. No OAS refers to OAS_MID (0.319 MAE). Liquidity contains log_volume, bd_px_spread, and quote_age_seconds (0.323 MAE). Bond features are DURATION_MID, CONVEXITY_MID, COUPON, and ttm_years (0.342 MAE). Full features represent the model with no absent features (0.290 MAE).

Feature ablation uncovered no obvious noise within the most important features (Fig. 9), with MAE increasing as features are removed. Both OAS and liquidity features appear to have similar weight on the model, as MAE increased to a nearly similar amount of around 0.32, while without bond features, this increased to around 0.34 MAE. From this, we assume that these features are all important to the model and should be evaluated when looking at errors. Without these features, predicted values differ more from the targeted absolute error, and thus it is harder to evaluate when BondDroid is wrong.

Table 1

Average Feature Value by Cluster

	bd_px_spread	bd_mid_yield	quote_age_seconds	log_volume	OAS_MID	SOT_MID	COUPON	ttm_years
--	--------------	--------------	-------------------	------------	---------	---------	--------	-----------

CLUSTER								
0	-0.5457	-0.2821	-0.0056	-0.0689	-0.4095	-0.2391	0.0374	3.9822
1	1.9801	1.2352	0.1384	-0.6534	2.5116	1.2481	0.0661	7.5052
2	0.8386	0.1072	-0.0971	0.4428	-0.0645	-0.0034	0.0465	23.9540
3	0.0974	0.0559	0.0218	0.0721	0.1218	0.0492	0.0544	6.9105

Table 2

Feature Deviation From Average Value

Feature	bd_px_spread	bd_mid_yield	quote_age_seconds	log_volume	OAS_M ID	SOT_M ID	COUPON	ttm_years	CLUSTER
Value	-0.7244	-0.4461	1.8022	-1.3580	-0.5607	-0.4224	0.0325	1.0842	0
Deviation	-0.1787	-0.1640	1.8078	-1.2891	-0.1513	-0.1834	-0.0049	-2.8980	
Value	-0.1352	-0.1679	1.6737	2.0845	-0.4545	-0.2400	0.0205	25.4565	2
Deviation	-0.9738	-0.2751	1.7708	1.6418	-0.3900	-0.2366	-0.0260	1.5026	
Value	0.3869	0.0034	-0.4777	0.8029	-0.2390	-0.0864	0.0720	14.1383	3
Deviation	0.2895	-0.0526	-0.4995	0.7307	-0.3608	-0.1356	0.0176	7.2277	

Note: Values are selected from separate examples in the top fifty failures in all of BondDroids predictions. Predictions were sorted by the absolute error of BondDroids predictions. Absolute residual for our predictions was cluster 0: 98.4118, cluster 2: 56.6931, cluster 3: 26.3323

Post-ablation analysis, failure analysis was conducted to assess some of the top failing predictions, both on BondDroid's side and our side. Table 1 describes the average feature value of the top eight features determined to cause high importance in our modeling. From this, we can see that while some features may be relatively similar, clusters tend to have drastically different means under specific conditions. This includes ttm_years (time to maturity) for cluster 2 at 23.9540 years, compared to cluster 0's 3.9822 years, and cluster 1 and 3's 7.5052 and 6.9105, respectively. On the other hand, certain features had near identical features, such as with coupon, where we see 0.0374, 0.0661, 0.0465, and 0.0544, for clusters 0-3, respectively. This represents a wide variation in data, and indicates that while certain features may be heavily varied, no cluster is fully different from another in every feature.

Knowing this, we took three separate predictions, all of which failed in different degrees, and which belonged to separate clusters (Table 2). The data representing cluster 0 had the largest failure in our modeling and had an absolute residual of 98.4118. In this data, we see little variation from the mean of the cluster, with the exception of quote age, log volume, and time to

maturity, which all have a difference ranging from $|1-3|$. As this residual error is so large, and the values are near to the average expected subtracted from those which rank at the top of the importance, this seems to be a systematic error (Fig. 8). From examining other means in clusters, it seems the model generalized the quote age, using this as the primary weight, in turn providing a high residual. In the example from cluster 2, we see similar results in the deviations, where several features are at a near-zero deviation from the mean. Examining these deviations in relation to the features with the top importance in XGBoost, we see that many of the features with a higher absolute deviation are not present (Fig. 8). Because of this, we may assume this error is due to random error. Finally, the example from cluster 3 is the most different from the others, with almost every deviation of features being near zero, except for time to maturity at 7.2277 years (Table 2). This feature is not on top of feature importance for XGBoost, as well as appearing as an outlier for time to maturity for the cluster. Due to these factors, this prediction appears to be an edge case, rather than something systematic or random.

While all of these examples are on the very edge of errors, as absolute error for BondDroid in these examples ranged from 98.5658 in cluster 0, to 32.4097 in cluster 3, there are ways to mitigate this error. One potential improvement would be the introduction of more rigorous outlier management during data preparation. Because several extreme residuals appear to be associated with strange market conditions, filtering or separately modeling these observations may further reduce their influence on training. In addition, the systematic failure observed in cluster 0 suggests that the model places excessive weight on quote age. This may be addressed by hyperparameter tuning and regularization, reducing the tendency of XGBoost to rely on a subset of predictive features.

Feature engineering may also be able to improve performance. Several market characteristics, such as interactions between quote age, liquidity, and time to maturity, are not explicitly represented in the features. Creating better interaction variables may allow the model to better capture nonlinear dynamic markets, which may contribute to pricing errors. Furthermore, because cluster 3 appears to represent a maturity-driven edge case, specialized features that instead represent maturity buckets may improve performance for long bonds.

Finally, a two-stage modelling approach may be beneficial for modelling. An anomaly detection model could identify observations that differ from the average population, allowing these bonds to be modeled separately from normal observations. Such an approach may be able to reduce how harshly rare edge cases impact modeling, while improving the predictive accuracy.

Discussion

Unsupervised learning demonstrated that BondDroid predictions can be separated into four distinct pricing regimes (Fig. 3). Through comparing HDBSCAN and GMMs, we were able to discover four distinct regimes that cover different aspects of data. While clusters were not fully isolated, it is clear that GMM was successful at isolating data to discover these regimes. Although HDBSCAN indicated only two clusters were present, these four regimes proved useful in our analysis and were successful at analyzing BondDroid further. When these clusters were examined on mean values from features, we saw that the values of these features tend to be somewhat distinct, especially regarding where features have more weight within a cluster (Table 2). Because of this, we learned that while clusters may have presented with noise, and potentially a lack of boundaries in places, these clusters have significance, as features inside indicate

differentiation that may drive the clustering itself.

As such, this difference within clusters was something that surprised us. While it is to be expected that clusters act differently, HDBSCAN treated the data as two clusters, with around 11% of the data being represented as noise. This was paired with using GMM, which we were able to determine that four regimes were suitable for clustering. As HDBSCAN treated these as only two clusters, it was not expected to see such a wide variance between these four clusters. However, data indicate that not only do these clusters have largely different fingerprints (Fig. 2), but also errors (Fig. 4), and mean deviation (Table 2) within the clusters are largely different from each other as well, indicating that rather than HDBSCAN's two-cluster approach, our four-cluster finding from GMM is more correct.

Due to the size of the data, the primary challenge presented for clustering was fitting our large amount of data into the models. As we had ~8.5M records, this task would have been large and wasted a lot of time doing so. To supplement this, when performing clustering on the entire dataset, instead of performing KMeans clustering on the entire dataset at once, we performed batch analysis using MiniBatchKMeans to scale to the whole dataset. This allowed us to save time and energy computationally as opposed to determining clusters using K-Means on the entire span of records, which would take much longer.

Although we approached unsupervised learning from several angles, including ways to limit computational expense, there is a possibility that it left out important features, variables, or rows of data altogether, which could have hampered clustering, and is something that could be improved in the future. For future modeling, this may be solved with solutions such as cloud computing to better handle computationally expensive data, or modules such as PySpark, where needed, to better cluster large sets of data. These methods could both be applied in places such as applying K-Means to our data, where it would otherwise be intense and hard to run, and these processes would make it easier to perform. Doing so, our results potentially may have resulted in less noise or more distinct clusters, as data is not only scaled and approximated, but instead fit on the full set of records.

Not only have we deduced pricing regimes within BondDroid and varying levels of error within them, but through supervised learning, we have also deduced that what causes these errors is dependent on the pricing regime of a bond. Using XGBoost, we were able to create a model predicting the absolute pricing error of BondDroid, with an MAE of only 0.29 (Fig. 5). This metric was chosen as pricing error was not a result predicted by BondDroid, and is instead an engineered feature; as such, we chose to look at how far from this feature our predictions lie. Knowing that our predictions are close to those of BondDroid, we were able to examine feature weights determining these predictions as a proxy for how BondDroid predicts prices. Through looking at feature weights, we found that not only do feature weights vary wildly, but they vary based on the cluster a bond belongs to (Fig. 8). Alongside this, looking at bonds sans clusters, these feature weights are different, with weights appearing on top of weights which do not appear in the top of clustered bonds (Fig. 7).

Many of these errors were expected to be standout cases and thus statistically insignificant. However, feature weights appear adjusted on clusters (Fig. 8), and examples of records within clusters appear to follow these weights, and as such provide a claim that these errors follow the regime (Table 2). While there may be such a case where BondDroid instead does not recognize

clusters, and thus models the pricing differently, our clustering and feature weights imply if this is happening, regimes are predicted behind the scenes either way. Although we saw feature weights were dependent on regime, there may be some guidance in error due to extreme conditions or outliers, as while features appear differentiated from the mean in a cluster, the records with the highest error in clusters appear roughly the same. Because of the way data is structured, this may also be a factor of the same bond being represented multiple times, as a bond may be traded multiple times by different people, and get similar errors, on different identifiers.

The largest challenge in supervised learning was likewise the amount of data we were processing. Not only was this extremely computationally intensive and took nearly an hour for certain models to train, but this data also took a long time to process and combine. To solve this, this data was split, making it more manageable for training, as well as preventing leakage, but also, data was extensively preprocessed, including using packages like Polars and DuckDB for efficient querying. To interpret failures, model importance was examined alongside the examination of averages of clusters, and data to better estimate and understand failures in the model and data itself.

Future work may expand on our solution by implementing more rigorous solutions to eliminate outliers and reduce edge cases before modeling occurs. More robust modeling may also take place, expanding on what we did now, such as with larger samples of hyperparameters we were unable to test due to the computational expense, or robust multilayered models, to better understand failures and where the BondDroid system performs well. While it may not be possible, utilizing the BondDroid model itself and examining failures directly would provide more exact direct comparisons than predicting based on the predictor. Alongside this, future work may revolve around mitigating duplicates of bond trades made on certain dates. This would need to be done carefully, however, as we are unsure how BondDroid makes its predictions, and thus any attempt to mitigate these may remove the strength that BondDroid otherwise would use.

Ethical Considerations

In general, one of the largest ethical dilemmas arises from how the data is structured and collected. Because we use real financial data about transactions with bonds, identifiers like Legal Entity Identifiers (LEI), FIGI, and CUSIP are all present within the data. There are several public processes where people are able to discover who registered for identifiers; however, this typically results in corporations and their addresses. Although this is the case, it may still be possible for other information to present itself, and in this case, it constitutes an issue where people's personal information is at risk. The best way to mitigate this is to instead remove features, either by not downloading them with data if possible, or not providing this information in post-processing where possible. Without unnecessary identification, it may reduce the risk of personal information leakage and allow for better privacy for people.

Additionally, ethical considerations may arise from interpreting machine learning results. In the unsupervised portion of the project, clustering is used to identify groups of bonds and market conditions associated with pricing regimes. While we assume these conditions to be accurate, they must remain descriptive rather than causal. Although membership of a specific cluster may be associated with higher or lower error rates. Clustering alone cannot determine why these errors occur. Identified groups all only represent patterns within data, and should not be viewed as a definitive explanation for grouping or model behavior. Alongside this, it should be taken

into account that cluster size may play a role in average errors, as larger clusters may affect error rates due to more data being present.

Similarly, supervised learning is meant to predict BondDroid's absolute pricing error and is used to determine features at play in prediction accuracy. Because we are unable to use BondDroid's model and thus cannot examine the exact structure, the models we use serve as stand-ins for predicting error, and factors that influence predictions. Consequently, both the model outputs and feature importance metrics should be viewed as correlational rather than causal relationships. While the models may perform well and give valuable insight, it does not definitely explain how BondDroid makes decisions internally, and instead must be used as assumptions.

Finally, there is the possibility that readers and users overgeneralize findings beyond the time period, market features, and bonds present within the dataset. Because we used a limited period of data dating back in time, or forward has the potential to behave vastly differently, and may change due to several factors influenced by economic conditions or general trading behaviors. As such, results should be viewed within the lens of the timeframe, and further analysis must be done if BondDroid is updated with expanded or newer models, or if new data is to be analyzed.

Statement of Work

Carter Pasternak: Led supervised learning and contributed to the final analysis. Interpreted final supervised and unsupervised results and wrote the main report.

Christian Goelz: Led exploratory data analysis and data cleaning. Designed and presented updates in standup meetings.

Transaint Gau: Led the unsupervised learning portion of the project and coordinated the proposal.

Shared responsibilities: All team members contributed to project scoping, code review, GitHub repository organization, final visualizations, written interpretation, and final report review/editing.

Bibliography

- Daly, M., Liu, X. (Amber), & Zuller, J. (2024, November 13). *Corporate bond pricing and trading: Predicting future prices and machine learning*. Stevens Financial Systems Center.
[Stevens Financial Systems Center page](#)
- Cabrol, A., Drobetz, W., Otto, T., & Puhan, T. X. (2024). *Predicting corporate bond illiquidity via machine learning*. *Financial Analysts Journal*, 80(3), 103–127.
<https://doi.org/10.1080/0015198X.2024.2350952>
- Feng, G., He, X., Wang, Y., & Wu, C. (2025). *Predicting individual corporate bond returns*. *Journal of Banking & Finance*, 171, 107372.
<https://doi.org/10.1016/j.jbankfin.2024.107372>
- 7 Chord. (2025). *7 Chord bond prices and liquidity indicators (top liquid bond prices & liquidity indicators present): Documentation*. Dewey Data.
<https://app.deweydata.io/data/7-chord/bond-prices-and-liquidity-indicators/top-liquid-bond-prices-liquidity-indicators-present/documentation>
- Financial Industry Regulatory Authority (FINRA). (2026). *FINRA data vendor information*.
<https://wrds-www.wharton.upenn.edu/pages/about/data-vendors/finra/>

Appendix

The following list of features was established through feature engineering and is the basis for all further analysis, via both unsupervised and supervised learning. This list includes both features and data types.

Aligned panel:

FIGI – string

cusip – string

trace_price – float 64-bit

trace_yield – float 64-bit

trade_volume – float 64-bit

trade_side – string

trade_ts – datetime[us]

REGION – string

MARKET_SEGMENT – string

bd_ts – datetime[us]

bd_mid_px – float 32-bit

bd_mid_yield – float 32-bit

bd_px_spread – float 32-bit

AVG_DAILY_TRADE_COUNT_60 – float 64-bit

TIMES_TRADED_60 – float 64-bit

TIME_SINCE_LAST_TRADE – string

BID_OAS – float 64-bit

ASK_OAS – float 64-bit

BID_SOT – float 64-bit

ASK_SOT – float 64-bit

DURATION_MID – float 64-bit

CONVEXITY_MID – float 64-bit

COUPON – float 64-bit

MATURITY – datetime[us]

BONDDROID_SECTOR – string

BONDDROID_INDUSTRY – string

TRIGGER_TYPE – string

is_snapshot – boolean

bd_pricing_failure – boolean

quote_age_seconds – float 32-bit

Additions in ML-ready panel:

target_abs_error – float 32-bit

log_volume – float 32-bit

OAS_MID – float 32-bit

SOT_MID – float 32-bit

REGION_East Asia & Pacific – unsigned 8-bit

REGION_Europe and Central Asia – unsigned 8-bit

REGION_Latin America and Caribbean – unsigned 8-bit

REGION_North America – unsigned 8-bit

REGION_South Asia – unsigned 8-bit

MARKET_SEGMENT_DM Corporate – unsigned 8-bit

MARKET_SEGMENT_EM Corporate – unsigned 8-bit

BONDDROID_SECTOR_Accommodation and Food Services – unsigned 8-bit

BONDDROID_SECTOR_Administrative and Support and Waste Management and Remediation Services – unsigned 8-bit

BONDDROID_SECTOR_Arts, Entertainment, and Recreation – unsigned 8-bit

BONDDROID_SECTOR_Construction – unsigned 8-bit

BONDDROID_SECTOR_Finance and Insurance – unsigned 8-bit

BONDDROID_SECTOR_Health Care and Social Assistance – unsigned 8-bit

BONDDROID_SECTOR_Information – unsigned 8-bit

BONDDROID_SECTOR_Management of Companies and Enterprises – unsigned 8-bit

BONDDROID_SECTOR_Manufacturing – unsigned 8-bit

BONDDROID_SECTOR_Mining, Quarrying, and Oil and Gas Extraction – unsigned 8-bit

BONDDROID_SECTOR_Professional, Scientific, and Technical Services – unsigned 8-bit

BONDDROID_SECTOR_Real Estate and Rental and Leasing – unsigned 8-bit

BONDDROID_SECTOR_Retail Trade – unsigned 8-bit

BONDDROID_SECTOR_Transportation and Warehousing – unsigned 8-bit

BONDDROID_SECTOR_Utilities – unsigned 8-bit

BONDDROID_SECTOR_Wholesale Trade – unsigned 8-bit

BONDDROID_INDUSTRY – unsigned 8-bit

TRIGGER_TYPE_ats1_primary – unsigned 8-bit

TRIGGER_TYPE_trace_primary – unsigned 8-bit